Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Sabor & Arte

Especificação dos Requisitos de Software

Para o Sistema de Gerenciamento de Buffet e Eventos

Versão 1.0

Autora: Alany Souza
Orientador: Prof. Hudson Neves
Disciplina: Projeto Integrador

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Histórico da Revisão

Data	Versão	Descrição	Autor
22/out/26	1.0	Versão inicial: levantamento completo a partir do código-fonte e conferência de tudo com o código (rotas, telas, banco e regras), com correção das divergências listadas no Apêndice C.	Alany Souza

Índice Analítico

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Especificação dos Requisitos de Software

1. Introdução

A introdução apresenta uma visão geral desta Especificação dos Requisitos de Software (ERS) do Sabor & Arte: finalidade, escopo, definições, referências e organização do documento.

1.1 Finalidade

Este documento apresenta a Especificação dos Requisitos de Software (ERS) do Sabor & Arte, um sistema web de gerenciamento de buffet e organização de eventos. A ERS descreve o comportamento externo do sistema, os requisitos funcionais e não funcionais, as restrições de design e as informações necessárias para que o sistema seja projetado, implementado, testado e avaliado.

O documento destina-se à autora do projeto (Alany Souza), ao orientador (Prof. Hudson Neves) e aos avaliadores da disciplina Projeto Integrador. O Sabor & Arte é um projeto construído para fins acadêmicos.

1.2 Escopo

O Sabor & Arte é um sistema web, desenvolvido individualmente, que centraliza a gestão de uma empresa de buffet e dos eventos por ela atendidos. O sistema possui duas áreas:

- **Painel administrativo** (pasta adm/): dashboard, clientes, eventos, orçamentos, pedidos e serviços (itens do cardápio). A gestão de funcionários consta do README, mas ainda não possui tela (Backlog).
- **Portal do cliente** (pasta public/): página inicial, cadastro, login, dashboard do cliente, cardápio, pacotes, criação de evento, confirmação de pedido, acompanhamento do pedido e do orçamento, mensagens e pagamento.

Esta ERS cobre o sistema completo. Os recursos ainda não implementados estão identificados como Backlog. Estão fora do escopo desta versão: emissão de nota fiscal, aplicativo móvel nativo, controle de estoque de insumos, gestão de escala de funcionários, exportação em PDF, notificações automáticas, integração real com gateway de pagamento, hash de senhas (bcrypt) e autenticação por token JWT.

1.3 Definições, Acrônimos e Abreviações

Termo	Definição
ERS / SRS	Especificação dos Requisitos de Software.
RF / RNF / RN / RD	Requisito Funcional / Requisito Não Funcional / Regra de Negócio / Restrição de Design.
CRUD	Criar, Ler, Atualizar e Excluir registros.

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Termo	Definição
API REST	Interface de comunicação HTTP entre frontend e backend, com dados em JSON.
JSON	JavaScript Object Notation, formato de troca de dados.
JWT	JSON Web Token, padrão de autenticação por token (Backlog).
bcrypt	Algoritmo de hash de senhas (Backlog).
Serverless	Execução de código sob demanda (usado na Vercel).
Pool	Conjunto de conexões reutilizáveis ao banco de dados.
Seed	Dados iniciais inseridos automaticamente.
Buffet	Empresa prestadora de serviços de alimentação para eventos.
Evento	Celebração contratada por um cliente (casamento, aniversário, formatura etc.).
Item / Serviço	Produto do cardápio (comida, bebida ou serviço) com valor unitário.
Orçamento	Proposta formal de valor para um evento, composta por itens e quantidades.
Pedido	Solicitação do cliente com itens, total e status de acompanhamento.
Admin	Usuário com perfil administrativo (usuario.tipo = 'admin').
Funcionário	Usuário interno com acesso ao painel (usuario.tipo = 'funcionario').
Cliente	Usuário final que contrata os serviços (usuario.tipo = 'cliente').
PIX	Meio de pagamento instantâneo brasileiro.
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018).
Backlog	Requisito previsto para versão futura, não implementado nesta versão.
TBD	To be determined: item ainda a definir.

1.4 Referências

Documento / fonte	Descrição
server.js	Backend principal; fonte primária dos endpoints e regras de negócio (1917 linhas).
db.js	Módulo de conexão com o banco via pool mysql2.
SABOR BUFFET SQL.sql	Script do banco buffet_db com o esquema e dados de exemplo.
scripts/migrate.js	Script de criação do banco a partir do arquivo SQL.

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Documento / fonte	Descrição
package.json / vercel.json / .env.example	Dependências, configuração de deploy na Vercel e modelo das variáveis de ambiente.
README.md	Documentação principal do repositório, com instalação e visão geral.
Repositório GitHub	https://github.com/AlanySouza2203/Sabor-Arte (consultado em 22/out/2026).
Aplicação publicada	https://sabor-arte-gamma.vercel.app
IEEE Std 830-1998	IEEE Recommended Practice for Software Requirements Specifications; base estrutural deste documento.
Lei nº 13.709/2018	Lei Geral de Proteção de Dados Pessoais (LGPD), Brasil.

1.5 Visão Geral

A Seção 2 descreve o produto de forma geral: perspectiva, funções, perfis de usuário, restrições, suposições e subconjuntos de requisitos. A Seção 3 contém os requisitos específicos: funcionais (3.1), não funcionais (3.2 a 3.5), restrições de design (3.6), ajuda e documentação (3.7), componentes adquiridos (3.8), interfaces (3.9), licenciamento (3.10), observações legais (3.11) e padrões aplicáveis (3.12). A Seção 4 reúne apêndices informativos.

Convenções: os verbos "deve" e "deverá" indicam requisito obrigatório; "pode" indica opcional. Cada requisito funcional informa Prioridade (Essencial, Importante ou Desejável) e Situação (Escopo atual ou Backlog).

2. Descrição Geral

Esta seção descreve os fatores gerais que afetam o produto e seus requisitos. Ela não define requisitos específicos; fornece a base para os requisitos detalhados na Seção 3.

2.1 Perspectiva do produto

O Sabor & Arte é um sistema web independente, organizado em três camadas: interface web (HTML, CSS e JavaScript), API REST (Node.js e Express) e banco de dados relacional (MySQL/MariaDB, banco buffet_db). O frontend consome a API por requisições HTTP em JSON; a API acessa o banco por meio do driver mysql2 (módulo db.js). O mesmo servidor Express entrega as páginas estáticas (public/, adm/ e img/) e a API. A aplicação é publicada na Vercel, com api/index.js como ponto de entrada serverless, para onde o vercel.json reescreve todas as rotas.

Componente	Tecnologia	Arquivos
Frontend: painel administrativo	HTML, CSS e JavaScript	adm/

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Componente	Tecnologia	Arquivos
Frontend: portal do cliente	HTML, CSS e JavaScript	public/, img/
Backend / API REST	Node.js e Express 5.2.1	server.js, api/index.js
Banco de dados	MySQL / MariaDB (mysql2 3.22.0)	db.js, SABOR BUFFET SQL.sql, scripts/migrate.js
Implantação	Vercel (serverless)	vercel.json

2.2 Funções do produto

- Cadastrar e autenticar clientes, administradores e funcionários pelo mesmo login unificado.
- Consultar o cardápio, escolher itens ou pacotes e configurar um evento.
- Confirmar pedidos e acompanhar seus status.
- Registrar o método de pagamento (PIX, crédito ou débito), na interface.
- Gerenciar clientes, eventos, serviços, orçamentos e pedidos no painel administrativo.
- Oferecer visão geral do negócio no dashboard administrativo.
- Inicializar automaticamente as tabelas usuario e item_servico, com usuários e itens padrão.
- Criar automaticamente evento e orçamento ao cadastrar um novo cliente pelo painel administrativo.
- Sincronizar o status de pedidos e orçamentos nos dois sentidos.

2.3 Características dos usuários

Usuário	Descrição	Acesso
Administrador (tipo = 'admin')	Gestor do buffet; gerencia todos os módulos.	Painel administrativo completo (/adm/).
Funcionário (tipo = 'funcionario')	Equipe operacional. No código atual, o dashboard aceita funcionário; as telas de clientes, eventos e orçamentos exigem perfil admin (verificação no navegador). Permissões a definir (TBD).	Painel administrativo (parcial).
Cliente (tipo = 'cliente')	Pessoa ou empresa que contrata eventos; perfil técnico variado.	Portal do cliente (/public/).

2.4 Restrições

As restrições de design estão detalhadas na Seção 3.6. Em resumo: senhas em texto plano (sem bcrypt), sem autenticação JWT, sem paginação, CSS embutido nos HTML, banco MySQL obrigatório, endpoint de debug exposto em produção e admin_id fixo = 1 para clientes cadastrados pelo portal.

2.5 Suposições e dependências

- Os usuários acessam o sistema por navegador moderno com suporte a ES6+ e Fetch API.
- Existe um banco MySQL/MariaDB acessível ao backend, configurado por variáveis de ambiente.
- A Vercel mantém suporte a funções serverless em Node.js.
- A integração com gateway de pagamento, e-mail e WhatsApp depende de serviços de terceiros ainda não definidos.
- Ícones e fontes são carregados de CDNs externas (jsDelivr, cdnjs, unpkg e Google Fonts).

2.6 Subconjuntos de requisitos

Subconjunto	Requisitos
Núcleo implementado (escopo atual)	RF01 a RF03, RF05 a RF07, RF09, RF15, RF16, RF18 a RF21; RF04 e RF08 parcialmente (somente interface)
Segurança (Backlog, alta prioridade)	RF12, RF13, RF14
Evoluções (Backlog)	RF10, RF11, RF17, RF22 a RF28

3. Requisitos Específicos

Esta seção contém os requisitos de software em nível de detalhe suficiente para projeto e teste. Não é utilizada modelagem de casos de uso; os requisitos estão em linguagem natural, com identificadores únicos.

3.1 Funcionalidade

Os requisitos funcionais estão organizados em dois grupos: portal do cliente (RF01 a RF11) e painel administrativo e segurança (RF12 a RF28), seguidos das regras de negócio.

3.1.1 RF01: Cadastro de cliente

Endpoint: POST /usuarios **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 764–840; tabelas usuario e cliente

O sistema deve permitir que um visitante se cadastre como cliente informando nome, telefone, e-mail e senha. O cadastro deve:

- validar que nome, e-mail e senha estão preenchidos e, para o perfil cliente, que o telefone também está;
- criar o registro na tabela usuario (nome, e-mail, senha em texto plano, tipo = 'cliente'); o telefone não é gravado em usuario, apenas em cliente;
- criar o registro na tabela cliente (nome, telefone, e-mail, usuario_id, admin_id = 1);
- rejeitar e-mails já cadastrados com HTTP 400 e a mensagem "E-mail já cadastrado" (RN06);

- redirecionar para login.html após o cadastro (o login não é automático).
- Limitação: a rota aceita tipo = "funcionario" sem autenticação, o que permite criar contas com acesso ao painel (ver Apêndice C).

3.1.2 RF02: Login de usuário

Endpoint: POST /login **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 681–757; tabela usuario

O sistema deve permitir que clientes, administradores e funcionários se autentiquem com e-mail e senha pela mesma tela (public/login.html). Em caso de credenciais inválidas, deve retornar resposta genérica (success: false, com HTTP 200), sem indicar qual campo está incorreto. Após o login, o frontend guarda o usuário no navegador (localStorage) e redireciona: admin ou funcionário para /adm/dashboard.html; cliente para /public/dashboard_cliente.html. Observação: se o e-mail informado for admin@email.com ou carlos@email.com e o usuário não existir, o login recria esses usuários padrão (senha "123").

3.1.3 RF03: Consulta ao cardápio

Endpoint: GET /itens ou GET /itens?categoria=X **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 102–145; tabela item_servico

O sistema deve listar os itens do cardápio exibindo nome, tipo, categoria, valor e imagem. O cliente pode filtrar por categoria. Se a tabela estiver vazia, o sistema insere 5 itens padrão (na inicialização e também na própria consulta). Limitação: itens inativos também são retornados, pois não há filtro por status = 'ativo' (RN08).

3.1.4 RF04: Seleção de pacotes de buffet

Endpoint: Nenhum (frontend estático) **Prioridade:** Importante **Situação:** Escopo atual (interface) / Backlog (backend) **Origem:** public/pacotes.html

O sistema deve permitir ao cliente escolher um pacote de buffet. A tela public/pacotes.html existe, mas não há tabela de pacotes no banco nem endpoints; a composição é estática no HTML (TBD).

3.1.5 RF05: Criação e configuração de evento

Endpoint: POST /eventos, GET /eventos?cliente_id=X **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 514–629; tabelas evento e evento_item

O sistema deve permitir ao cliente autenticado criar um evento informando tipo, data, local e número de convidados. Tipo, data e local são obrigatórios (HTTP 400 se faltarem). O evento é vinculado ao cliente (RN01) e nasce com status 'Planejado'. O número de convidados é opcional (padrão = 0).

3.1.6 RF06: Confirmação de pedido

Endpoint: POST /pedidos **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 847–931; tabela pedidos

O sistema deve exibir um resumo do pedido e permitir sua confirmação. Ao confirmar, registra o pedido vinculado ao cliente e ao evento, com status 'Pendente', armazenando os itens como JSON serializado no campo pedidos.itens. O total é calculado automaticamente se não informado.

3.1.7 RF07: Acompanhamento de pedido e orçamento

Endpoint: GET /meu-pedido/:clienteId, GET /cliente/orcamento-completo/:clienteId **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 977–1005 e 1728–1812; tabelas pedidos e orcamento

O sistema deve permitir ao cliente consultar, na área "Meu Pedido", a situação do seu pedido (status, total, evento) e do seu orçamento (status, valor, itens, status de pagamento). A consulta completa retorna orçamento, itens, evento e cliente em uma única chamada. Estão disponíveis ainda GET /cliente/orcamento/:clienteId e GET /cliente/orcamento/:orcamentoId/itens.

3.1.8 RF08: Pagamento do pedido

Endpoint: Nenhum implementado no backend **Prioridade:** Essencial **Situação:** Interface: escopo atual / Registro no banco: Backlog **Origem:** public/pagamento.html; tabela pagamento (sem endpoints)

O sistema deve permitir ao cliente escolher o método de pagamento (PIX, crédito ou débito). A interface existe (public/pagamento.html) e a tabela pagamento está criada no SQL, mas não há endpoints para registro de pagamento nesta versão; a tela trabalha apenas com dados do navegador. O valor pago não pode ultrapassar o total do orçamento (RN05, Backlog).

3.1.9 RF09: Dashboard do cliente

Endpoint: GET /cliente/orcamento-completo/:clienteId **Prioridade:** Importante **Situação:** Escopo atual **Origem:** public/dashboard_cliente.html

O sistema deve oferecer ao cliente uma área inicial reunindo informações de seus eventos, pedidos e orçamentos.

3.1.10 RF10: Histórico completo de pedidos do cliente

Prioridade: Desejável **Situação:** Backlog

O sistema deverá listar todos os pedidos realizados pelo cliente, com data, status e total, permitindo consultar o detalhe de cada um.

3.1.11 RF11: Montagem interativa de prato

Prioridade: Desejável **Situação:** Backlog

O sistema deverá oferecer um módulo de cardápio interativo em que o cliente monte seu prato. Detalhamento a definir (TBD).

3.1.12 RF12: Autenticação do administrador

Prioridade: Essencial **Situação:** Backlog

O painel administrativo deve exigir login de usuário com perfil administrador, mantendo sessão por token JWT com prazo de expiração. Sem autenticação válida, o usuário deve ser redirecionado à tela de login. Situação atual: não há tela de login administrativa separada (adm/login.html não existe) nem verificação de token; o painel apenas confere, no navegador, o usuário salvo após o login.

3.1.13 RF13: Controle de acesso e proteção de rotas

Prioridade: Essencial **Situação:** Backlog

As rotas REST administrativas (POST, PUT e DELETE de clientes, eventos, pedidos, orçamentos e itens) devem exigir token JWT válido no cabeçalho Authorization: Bearer, retornando HTTP 401 quando ausente ou inválido. Situação atual: todas as rotas são abertas.

3.1.14 RF14: Armazenamento seguro de senhas

Prioridade: Essencial **Situação:** Backlog

O sistema deve armazenar senhas somente na forma de hash (bcrypt) e nunca em texto puro. A verificação no login deve comparar a senha informada com o hash armazenado. Situação atual: senhas gravadas e comparadas em texto plano.

3.1.15 RF15: Dashboard administrativo

Prioridade: Importante **Situação:** Escopo atual **Origem:** adm/dashboard.html

O sistema deve apresentar ao administrador uma visão geral do negócio, com indicadores (a partir de pedidos, clientes e eventos) e acesso rápido aos módulos pelo menu lateral.

3.1.16 RF16: Gestão de clientes

Endpoint: GET /clientes, POST /clientes, PUT /clientes/:id, DELETE /clientes/:id **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 262–507; tabela cliente

O sistema deve permitir ao administrador cadastrar, consultar, editar e excluir clientes (nome, telefone e e-mail), mantendo a unicidade do e-mail (RN06). O cadastro administrativo cria automaticamente um evento padrão e um orçamento inicial vinculados ao cliente. A exclusão é bloqueada (HTTP 400) se o cliente possuir eventos vinculados (RN07). Limitação: GET /clientes retorna todos os clientes e ignora o parâmetro admin_id enviado pelo painel.

3.1.17 RF17: Gestão de funcionários

Prioridade: Importante **Situação:** Backlog

O sistema deve permitir ao administrador cadastrar, consultar e editar funcionários. Situação atual: adm/funcionarios.html não existe; funcionários são apenas registros com tipo = 'funcionario' na tabela usuario.

3.1.18 RF18: Gestão de eventos

Endpoint: GET /eventos, POST /eventos, DELETE /eventos/:id **Prioridade:** Essencial **Situação:** Escopo atual (listar, criar, excluir); editar: Backlog **Origem:** server.js linhas 509–675; tabela evento

O sistema deve permitir ao administrador listar, criar e excluir eventos, vinculando cada evento a um cliente e controlando seu status (Planejado, Em andamento, Finalizado ou Cancelado). A listagem aceita os filtros cliente_id e admin_id. Limitação: a edição de eventos (PUT /eventos/:id) não está implementada.

3.1.19 RF19: Gestão de serviços (itens do cardápio)

Endpoint: GET /itens, POST /itens, PUT /itens/:id, DELETE /itens/:id **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 97–256; tabela item_servico

O sistema deve permitir ao administrador cadastrar, editar e excluir itens do cardápio, informando nome, tipo (comida, bebida ou servico), categoria, valor e imagem. Limitação: ativar ou inativar itens pela API não funciona, pois PUT /itens/:id não atualiza o campo status.

3.1.20 RF20: Gestão de orçamentos

Endpoint: GET /orcamentos, GET /orcamentos/:id, POST /orcamentos, PUT /orcamentos/:id **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 1262–1689; tabelas orcamento e item_orcamento

O sistema deve permitir ao administrador criar orçamentos para eventos com itens e quantidades, calcular o total automaticamente (RN03), aprovar ou recusar o orçamento e acompanhar o status do pagamento. A criação exige evento e ao menos um item (RN02). A aprovação ou recusa do orçamento sincroniza o status do pedido vinculado (Aprovado: Confirmado; Recusado: Cancelado).

3.1.21 RF21: Gestão de pedidos

Endpoint: GET /pedidos, GET /meu-pedido/:clienteId, POST /pedidos, PUT /pedidos/:id/status, PUT /pedidos/:id/confirmar, DELETE /pedidos/:id **Prioridade:** Essencial **Situação:** Escopo atual **Origem:** server.js linhas 842–1260; tabela pedidos

O sistema deve permitir ao administrador listar pedidos e atualizar seu status. Os status previstos no banco são Pendente, Confirmado, Finalizado e Cancelado; o código também trata o status 'Preparando', que não consta do ENUM da tabela (ver Apêndice C). A atualização de status sincroniza o orçamento vinculado. A confirmação de pedido (PUT /pedidos/:id/confirmar) gera um orçamento formal com os itens do pedido. Limitação: GET /pedidos aceita apenas os filtros evento_id e cliente_id; o parâmetro admin_id enviado pelo painel é ignorado.

3.1.22 RF22: Paginação das listagens

Prioridade: Desejável **Situação:** Backlog

O sistema deverá paginar as listagens do painel administrativo para limitar o volume de dados por requisição.

3.1.23 RF23: Filtros por período e relatório de faturamento

Prioridade: Desejável **Situação:** Backlog

O sistema deverá permitir filtrar o dashboard por intervalo de datas e apresentar relatório de faturamento.

3.1.24 RF24: Escala de funcionários para eventos

Prioridade: Desejável **Situação:** Backlog

O sistema deverá permitir atribuir funcionários aos eventos.

3.1.25 RF25: Exportação de orçamentos em PDF

Prioridade: Desejável **Situação:** Backlog

O sistema deverá gerar orçamentos em PDF para envio direto ao cliente por WhatsApp ou e-mail.

3.1.26 RF26: Notificações automáticas

Prioridade: Desejável **Situação:** Backlog

O sistema deverá enviar notificações automáticas por e-mail e WhatsApp ao cliente quando o status do orçamento ou pedido for atualizado.

3.1.27 RF27: Integração com gateway de pagamento

Prioridade: Importante **Situação:** Backlog

O sistema deverá integrar-se a um gateway de pagamento para PIX e cartão, registrando identificador da transação, código PIX, gateway utilizado e resposta recebida. Os campos já estão previstos na tabela pagamento.

3.1.28 RF28: Mensagens entre cliente e buffet

Endpoint: Previstos pela interface: /mensagens, /cliente/enviar-mensagem, /cliente/mensagens/nao-lidas/count e /cliente/mensagens/marcar-lidas **Prioridade:** Desejável **Situação:** Interface: escopo atual / Backend: Backlog **Origem:** public/mensagens.html

O sistema deverá permitir a troca de mensagens entre o cliente e o buffet, vinculadas a um pedido, com contagem de mensagens não lidas. A tela public/mensagens.html existe e chama os endpoints indicados, mas nenhum deles está implementado em server.js e não há tabela de mensagens no SQL. Requisito incluído na versão 1.0.

3.1.29 Regras de negócio

Código	Regra	Situação
RN01	Todo evento deve estar associado a um cliente (evento.cliente_id obrigatório).	Implementado
RN02	Um orçamento deve conter pelo menos um item.	Validado na API (POST /orcamentos)
RN03	O total do orçamento é calculado a partir dos itens; subtotal = quantidade × valor.	Implementado
RN04	A confirmação do evento depende da aprovação do orçamento.	Não validada no código
RN05	O valor pago não deve ultrapassar o total do orçamento.	Sem endpoints de pagamento
RN06	O e-mail deve ser único entre usuários e entre clientes.	Constraint UNIQUE e tratamento de ER_DUP_ENTRY
RN07	A exclusão de cliente verifica eventos vinculados antes de prosseguir.	Verificação manual; bloqueia se há eventos
RN08	Somente itens com status ativo devem ser ofertados no cardápio.	GET /itens não filtra por status

3.2 Usabilidade

Esta seção contém os requisitos que afetam a usabilidade do sistema.

RNF-U01: Responsividade

As interfaces do portal do cliente (public/) devem ser responsivas em computadores, tablets e smartphones. Todas as telas possuem regras @media; a cobertura em cada tela do painel administrativo deve ser validada (TBD).

RNF-U02: Consistência de navegação

As telas devem manter padrão visual e de navegação consistentes, com menu lateral fixo no painel administrativo e menu principal persistente no portal do cliente.

RNF-U03: Mensagens claras

Erros de validação, de autenticação e de comunicação devem ser exibidos em linguagem clara ao usuário, sem expor detalhes técnicos do banco de dados. Situação atual: alguns endpoints ainda retornam o erro técnico do MySQL (por exemplo, o endpoint de debug).

RNF-U04: Fluxo de contratação objetivo

O cliente deve concluir a contratação em fluxo linear e curto: cadastro, login, cardápio/pacotes, evento, confirmação do pedido, acompanhamento e pagamento.

3.3 Confiabilidade

Os requisitos de confiabilidade do sistema são especificados a seguir.

RNF-C01: Integridade referencial

O banco de dados deve preservar a integridade entre usuários, clientes, eventos, itens, orçamentos, pedidos e pagamentos por meio de chaves estrangeiras com as cláusulas adequadas de CASCADE, conforme o script SQL.

RNF-C02: Tratamento centralizado de erros

A API deve possuir um middleware global de tratamento de erros que retorne resposta padronizada, não exponha stack trace ao cliente e registre os erros em log com data e hora. Situação atual: Backlog; não há middleware global e os erros são tratados rota a rota, com console.log.

RNF-C03: Consistência de valores monetários

Valores monetários devem ser armazenados com precisão decimal (DECIMAL(10,2)), e totais e subtotais devem ser calculados de forma consistente (RN03).

RNF-C04: Disponibilidade

O sistema publicado deve estar disponível em pelo menos 99% do tempo mensal, excluídas manutenções programadas, compatível com o SLA da Vercel (meta proposta; validar com o orientador).

RNF-C05: Cópias de segurança

A política de backup do banco em produção ainda precisa ser definida (TBD).

3.4 Desempenho

As características de desempenho do sistema são descritas a seguir.

RNF-D01: Tempo de resposta

Em condições normais de uso, listagens e consultas simples devem responder em até 2 segundos; operações compostas (criação de cliente com evento e orçamento) em até 5 segundos (metas propostas).

RNF-D02: Paginação

As listagens devem ser paginadas para limitar o volume de dados por requisição. Situação atual: Backlog; as listagens retornam todos os registros.

RNF-D03: Capacidade

O sistema deve suportar o uso simultâneo de algumas dezenas de usuários sem degradação perceptível, compatível com o porte acadêmico do projeto.

RNF-D04: Conexões com o banco em ambiente serverless

A conexão com o MySQL deve ser compatível com a execução serverless da Vercel, evitando o esgotamento de conexões. O db.js usa pool com até 10 conexões; o comportamento em serverless deve ser validado (TBD).

3.5 Suportabilidade

Esta seção indica os requisitos que melhoram a suportabilidade e a manutenibilidade do sistema.

RNF-S01: Organização do código

O código deve seguir a estrutura de pastas do projeto: adm/ (painel), public/ (portal), api/ (ponto de entrada serverless), scripts/ (migração), img/ (imagens), server.js e db.js na raiz.

RNF-S02: Configuração por variáveis de ambiente

Parâmetros de ambiente (PORT, DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME e DB_SSL) devem ser definidos em arquivo .env, com modelo em .env.example. Credenciais não devem ser publicadas no repositório; recomenda-se incluir um .gitignore com o .env, que hoje não existe no repositório.

RNF-S03: Criação e migração do banco

O esquema e os dados iniciais do banco devem poder ser criados a partir de SABOR BUFFET SQL.sql e do script scripts/migrate.js (npm run migrate). O server.js cria sozinho apenas as tabelas usuario e item_servico.

RNF-S04: Versionamento e backlog

O código deve ser versionado com Git e GitHub, e as evoluções devem ser registradas como issues no repositório.

RNF-S05: Documentação

O README deve manter instruções de instalação e execução. Documentação dos endpoints, diagrama entidade-relacionamento e capturas de tela ainda a adicionar (TBD).

RNF-S06: Testes

O projeto deve possuir testes que validem o funcionamento da API. Situação atual: Backlog; o script npm test executa apenas "node --check server.js" (verificação de sintaxe), sem testes automatizados.

3.6 Restrições de Design

Esta seção indica as restrições de design que representam decisões impostas e que devem ser obedecidas.

RD01: Frontend em HTML, CSS e JavaScript

A interface deve ser implementada com HTML5, CSS3 e JavaScript puro, sem framework de interface (React, Vue, Angular etc.).

RD02: Backend em Node.js e Express

A lógica de servidor deve ser implementada em Node.js com Express 5.x, expondo uma API REST stateless.

RD03: Banco MySQL/MariaDB

Os dados devem ser armazenados em banco relacional MySQL ou MariaDB, chamado buffet_db, acessado pelo driver mysql2. As consultas utilizam parâmetros preparados (?) para evitar SQL Injection.

RD04: Hospedagem na Vercel

A aplicação é implantada na Vercel, com api/index.js como ponto de entrada serverless e configuração em vercel.json.

RD05: Projeto individual acadêmico

O projeto é desenvolvido individualmente, no âmbito da disciplina Projeto Integrador, sob orientação do Prof. Hudson Neves, com prazos definidos pela disciplina.

3.7 Requisitos de Sistema de Ajuda e de Documentação de Usuário On-line

O sistema não prevê, nesta versão, sistema de ajuda ou manual de usuário on-line integrado à interface. O suporte ao uso é feito por mensagens claras de validação e erro (RNF-U03) e, para quem instala o sistema, pelo README.md do repositório. Um guia rápido para clientes e administradores é desejável e deve ser definido em versão futura (TBD).

3.8 Componentes Adquiridos

Não há componentes adquiridos comercialmente. O sistema utiliza exclusivamente bibliotecas e recursos de código aberto ou gratuitos. As dependências do servidor estão registradas no package.json; os recursos de interface são carregados por CDN.

Componente	Versão	Finalidade	Licença
Node.js	LTS	Ambiente de execução do backend	MIT
Express	5.2.1	Framework web para Node.js	MIT
mysql2	3.22.0	Driver de conexão com MySQL	MIT
cors	2.8.6	Middleware CORS para Express	MIT
Font Awesome Free	6.5.0	Biblioteca de ícones (CDN)	CC BY 4.0, OFL e MIT
Bootstrap Icons	1.11.3	Biblioteca de ícones (CDN)	MIT
Google Fonts (Inter)	n/d	Tipografia (CDN)	OFL
Lucide	latest	Ícones SVG (CDN unpkg)	ISC
jsonwebtoken	Previsto	Tokens JWT	MIT (Backlog)
bcrypt	Previsto	Hash de senhas	MIT (Backlog)

3.9 Interfaces

Esta seção define as interfaces que devem ser suportadas pelo aplicativo.

3.9.1 Interfaces do Usuário

Painel Administrativo (adm/)

Arquivo	Funcionalidade	Status
dashboard.html	Visão geral com indicadores e menu lateral	Implementado
clientes.html	Cadastro, edição e exclusão de clientes	Implementado
eventos.html	Listagem e gestão de eventos	Implementado
orcamentos.html	Criação, aprovação e recusa de orçamentos	Implementado
pedidos.html	Controle e atualização de status de pedidos	Implementado
servicos.html	CRUD de itens do cardápio	Implementado
login.html (administrativo)	Tela de login administrativa; o login é feito em public/login.html	Backlog (não existe)
funcionarios.html	Gestão de funcionários	Backlog (não existe)

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Portal do Cliente (public/)

Arquivo	Funcionalidade	Status
index.html	Página inicial (landing page) com tema escuro e acentos dourados	Implementado
cadastro.html	Formulário de registro de cliente	Implementado
login.html	Autenticação (único para todos os perfis)	Implementado
dashboard_cliente.html	Área personalizada do cliente	Implementado
cardapio.html	Exibição e seleção de itens por categoria	Implementado
pacotes.html	Seleção de pacotes (frontend estático)	Implementado
evento.html	Configuração do evento	Implementado
confirpedi.html	Revisão e confirmação do pedido	Implementado
meu_pedido.html	Acompanhamento com linha do tempo de status	Implementado
mensagens.html	Interface de mensagens	HTML existe / sem endpoints
pagamento.html	Seleção de método de pagamento	HTML existe / sem endpoint

3.9.2 Interfaces de Hardware

O sistema não define interfaces de hardware específicas. O acesso ocorre por navegador, em computadores, tablets e smartphones; o servidor é gerenciado pela Vercel e o banco de dados é um servidor MySQL/MariaDB.

3.9.3 Interfaces de Software

Interface	Descrição
Frontend e API	Requisições HTTP com dados em JSON, a partir das páginas em adm/ e public/.
API e banco de dados	Driver mysql2, módulo db.js, parâmetros DB_* em variáveis de ambiente.
Hospedagem	Vercel: vercel.json e api/index.js (função serverless).
Gateway de pagamento (previsto)	PIX e cartão; provedor a definir (RF27).
E-mail e WhatsApp (previstos)	Envio de notificações; provedores a definir (RF26).
Geração de PDF (prevista)	Biblioteca a definir (RF25).

Endpoints implementados em server.js:

Endpoint	Métodos	Finalidade
/itens, /itens/:id	GET, POST, PUT, DELETE	Itens do cardápio.
/clientes, /clientes/:id	GET, POST, PUT, DELETE	Clientes.
/eventos, /eventos/:id	GET, POST, DELETE	Eventos.
/login	POST	Autenticação.
/usuarios	POST	Cadastro de usuário/cliente.
/pedidos, /pedidos/:id/status, /pedidos/:id/confirmar, /pedidos/:id	GET, POST, PUT, DELETE	Pedidos.
/meu-pedido/:clienteId	GET	Pedido do cliente.
/orcamentos, /orcamentos/:id	GET, POST, PUT	Orçamentos.
/cliente/orcamento-completo/:clienteId	GET	Orçamento, itens, evento e cliente.
/cliente/orcamento/:clienteId e /cliente/orcamento/:orcamentoId/itens	GET	Orçamento e itens do cliente.
/debug/orcamento/:id	GET	Depuração (deve ser removido da produção).

3.9.4 Interfaces de Comunicação

A comunicação ocorre por HTTP/HTTPS, no padrão REST, com JSON. O servidor Express escuta na porta definida em PORT (o .env.example usa 3000) e a conexão com o MySQL usa DB_HOST, DB_PORT (exemplo: 3306), DB_USER, DB_PASSWORD, DB_NAME e DB_SSL. O CORS está habilitado para qualquer origem. Está previsto o uso do cabeçalho Authorization (Bearer) nas rotas protegidas (RF13).

3.10 Requisitos de Licenciamento

A licença do projeto ainda não foi definida (o README indica "a definir pela autora"; o package.json declara ISC). Enquanto nenhuma licença for incluída no repositório, não se deve presumir licença de código aberto. As bibliotecas utilizadas mantêm suas próprias licenças. O sistema não implementa mecanismo de licenciamento ou de controle de uso.

3.11 Observações Legais, de Copyright e Outras

O projeto foi construído para fins acadêmicos, é de autoria de Alany Souza e foi desenvolvido no âmbito da disciplina Projeto Integrador. O sistema trata dados pessoais (nome, telefone e e-mail) e, em uso real, deverá observar a LGPD. As imagens do cardápio padrão são carregadas de fontes externas; os direitos de uso devem ser verificados antes de qualquer uso comercial.

3.12 Padrões Aplicáveis

- IEEE Std 830-1998, para a estrutura desta ERS.
- Lei nº 13.709/2018 (LGPD), para o tratamento de dados pessoais.
- HTTP e REST para a API; SQL para o banco de dados.
- Recomendado: WCAG 2.1, nível AA, para acessibilidade das interfaces.

4. Informações de Suporte

Os Apêndices A, B e C são informativos e não fazem parte dos requisitos, exceto quando um requisito os cita expressamente.

4.1 Apêndice A: Rastreabilidade das user stories

US	Issue	Título	Situação	Requisitos
US01	#6	Autenticação e Sessão de Administrador	Aberta	RF12
US02	#11	Integração da Tela de Pedidos e Orçamentos	Aberta	RF20, RF21
US03	#7	Execução do Banco de Dados e Mapeamento ORM/SQL	Aberta	RNF-S03, RD03
US04	#8	CRUD de Serviços e Eventos	Aberta	RF18, RF19
US05	#1	Exportação de Orçamentos em PDF	Aberta	RF25
US06	#2	Notificação Automática por E-mail e WhatsApp	Aberta	RF26
US07	#3	Filtros Avançados e Busca no Dashboard	Aberta	RF23
US08	#12	Configuração do Ambiente de Desenvolvimento	Aberta	RNF-S02
US09	#13	Layout e Wireframe da Área Administrativa	Concluída	RNF-U01; 3.9.1
US10	#14	Arquitetura de Pastas e Ponto de Entrada Serverless	Concluída	RNF-S01, RD04
US12	#4	Módulo de Cardápio Interativo / Monte seu Prato	Aberta	RF11
US13	#5	Gestão de Funcionários e Escala para Eventos	Aberta	RF17, RF24
US14	#9	Tela de Login Administrativo	Aberta	RF12

US	Issue	Título	Situação	Requisitos
US15	#10	Tratamento de Erros e Logs no Servidor	Aberta	RNF-C02
US16	#16	Hash de Senhas com bcrypt	Aberta	RF14
US17	#17	Middleware JWT para Rotas Administrativas	Aberta	RF12, RF13
US18	#18	Middleware Global de Tratamento de Erros	Aberta	RNF-C02
US19	#19	Paginação nas Listagens do Painel Administrativo	Aberta	RF22, RNF-D02
US20	#20	Relatório de Faturamento no Dashboard Administrativo	Aberta	RF23
US21	#21	Integração com Gateway de Pagamento (PIX e Cartão)	Aberta	RF27
US22	#22	Histórico Completo de Pedidos do Cliente	Aberta	RF10
US23	#23	Script de Migração e Inicialização do Banco (migrate.js)	Aberta	RNF-S03

4.2 Apêndice B: Modelo de dados (banco buffet_db)

Tabela	Descrição	Campos principais
usuario	Contas de acesso (admin, funcionario, cliente).	id, nome, telefone, email (único), senha, tipo, created_at
cliente	Dados do cliente, vinculado a uma conta.	id, nome, telefone, email (único), usuario_id (único), admin_id
evento	Eventos contratados.	id, cliente_id, admin_id, tipo, data, local, convidados, status
item_servico	Catálogo de itens e serviços.	id, nome, tipo, categoria, valor, imagem, status, updated_at
evento_item	Itens escolhidos para cada evento.	id, evento_id, item_servico_id, quantidade
orcamento	Orçamento de um evento.	id, evento_id, admin_id, valor_total, status, status_pagamento, observacoes
item_orcamento	Itens do orçamento.	id, orcamento_id, item_servico_id, quantidade, valor, subtotal (calculado)
pedidos	Pedidos dos clientes.	id, cliente_id, evento_id, admin_id, itens (JSON), total, status, metodo_pagamento, status_pagamento

Sabor & Arte	Versão: 1.0
Especificação dos Requisitos de Software	Data: 22/out/26
ERS-SABORARTE-001	

Tabela	Descrição	Campos principais
pagamento	Pagamentos de orçamentos.	id, orcamento_id, metodo_pagamento, valor, parcelas, status, transacao_id, codigo_pix, gateway, resposta_gateway (JSON), data_pagamento

Campo	Valores (ENUM)
evento.status	Planejado, Em andamento, Finalizado, Cancelado
item_servico.tipo / status	comida, bebida, servico / ativo, inativo
orcamento.status	pendente, aprovado, recusado
orcamento.status_pagamento	pendente, processando, parcial, pago, cancelado, recusado
pedidos.status	Pendente, Confirmado, Finalizado, Cancelado
pedidos.status_pagamento	pendente, processando, pago, recusado
pagamento.status	pendente, processando, pago, recusado, cancelado
metodo_pagamento (pedidos e pagamento)	pix, credito, debito

4.3 Apêndice C: Itens a confirmar e divergências encontradas

Este apêndice reúne os pontos pendentes e as divergências encontradas na conferência da ERS com o código (versão 1.0). Deve ser removido da versão final entregue.

C.1 Divergências entre a ERS anterior (v1.0) e o código, já corrigidas nesta versão

- As telas adm/login.html e adm/funcionarios.html constavam como "Implementado", mas os arquivos não existem; passaram a Backlog (3.9.1).
- O login responde com HTTP 200 e success: false em caso de erro (não HTTP 401); RF02 ajustado.
- O seed dos 5 itens ocorre na inicialização e também dentro de GET /itens quando a tabela está vazia (RF03).
- O perfil funcionário não tem as mesmas permissões do admin: o dashboard o aceita, mas clientes, eventos e orçamentos exigem tipo admin (verificação no navegador); servicos.html não verifica o login (2.3).
- Font Awesome foi reclassificado de "Freemium" para a versão Free (licenças CC BY 4.0, OFL e MIT).
- Linhas duplicadas removidas das seções de definições (Orçamento) e referências (server.js).

- Incluídos o script npm test (apenas verificação de sintaxe), o requisito RNF-S06, o requisito RF28 (mensagens) e a ausência de .gitignore.

C.2 Pontos de atenção encontrados no código

Nº	Ponto	Observação / recomendação
1	Status "Preparando" fora do ENUM	PUT /pedidos/:id/status aceita qualquer texto e o código mapeia "Preparando" para pedidos e "preparando" para orçamentos, mas os ENUM de pedidos.status e orcamento.status não incluem esses valores; a gravação deve falhar no MySQL. Incluir os valores no SQL ou remover do código. RF21 foi ajustado.
2	Filtro admin_id ignorado	O painel chama /pedidos?admin_id= e /clientes?admin_id=, mas GET /pedidos só filtra por evento_id e cliente_id, e GET /clientes não filtra; todos os administradores veem todos os registros.
3	Possível erro em GET /pedidos?cliente_id=	A consulta junta pedidos e evento e usa "cliente_id" sem prefixo; como as duas tabelas têm essa coluna, o MySQL tende a retornar "coluna ambígua" (HTTP 500). Testar e corrigir com p.cliente_id e p.evento_id.
4	Criação de funcionário sem autenticação	POST /usuarios aceita tipo = "funcionario" de qualquer visitante, o que dá acesso ao dashboard. Restringir a criação a administradores (RF13).
5	Senhas e usuários padrão	Senhas em texto puro ("123") no SQL e no server.js; o login recria admin@email.com e carlos@email.com se não existirem. Remover antes de publicar com dados reais (RF14).
6	Controle de acesso só no navegador	O painel confere localStorage; as rotas da API são abertas. servicos.html não verifica o login (RF12, RF13).
7	Endpoint de debug	GET /debug/orcamento/:id está exposto e devolve dados e erros do banco. Remover da produção.
8	Endpoints de mensagens ausentes	mensagens.html chama 4 rotas que não existem em server.js (RF28).
9	PUT /itens/:id não grava status	Impede ativar/inativar itens (RF19, RN08).
10	Sem .gitignore	O repositório não possui .gitignore; incluir .env e node_modules para evitar publicar credenciais.
11	Logs verbosos	POST /orcamentos registra o corpo completo da requisição no console; reduzir em produção.